



POLITECHNIKA POZNAŃSKA

WYDZIAŁ INFORMATYKI I TELEKOMUNIKACJI
Instytut Informatyki

Praca dyplomowa magisterska

ODKRYWANIE ZALEŻNOŚCI MIĘDZY OBIEKTAMI W BAZIE DANYCH

Maria Nowicka, 151851

Promotor
dr hab. inż. Robert Wrembel, prof. PP.

POZNAŃ 2026

Tutaj będzie karta pracy dyplomowej;
oryginał wstawiamy do wersji dla archiwum PP,
w pozostałych kopiach wstawiamy ksero.

Abstract

Spis treści

Abstract	I
1 Wstęp	1
1.1 Cel i zakres pracy	1
1.2 Struktura pracy	2
2 Podstawy teoretyczne i przegląd literatury	3
2.1 Data lineage i data provenance	3
2.2 Struktura grafu lineage w zbiorze DLG-DG-23	3
2.3 Niekompletność grafów lineage — broken lineage	4
2.4 Wykrywanie brakujących krawędzi	4
2.5 Metryki ewaluacji	5
2.6 Wyniki systematycznego przeglądu literatury	5
2.6.1 Leveraging machine learning techniques for discovering broken lineage links	5
2.6.2 An open dataset of data lineage graphs	6
2.6.3 Prace kontekstowe	7
3 Problem i sposób jego rozwiązania	8
3.1 Charakterystyka danych	8
3.2 Scenariusze symulacji broken lineage	9
3.3 Zaimplementowane algorytmy	9
3.3.1 Heurystyki grafowe	9
3.3.1.1 Preferential Attachment	9
3.3.2 Klasyczne metody uczenia maszynowego	9
3.3.3 Zanurzenia sieciowe — Node2Vec	9
3.4 Architektura rozwiązania	9
4 Ocena rozwiązania	10
4.1 Wyniki eksperymentów	10
4.2 Analiza wyników	10

5 Uwagi końcowe	11
5.1 Podsumowanie	11
5.2 Kierunki dalszych badań	11
Literatura	12

Rozdział 1

Wstęp

Współczesne systemy bazodanowe przetwarzają dane za pośrednictwem złożonych potoków przetwarzania danych. W środowiskach korporacyjnych dane są wielokrotnie agregowane i przekształcane przez sekwencje zapytań, a wyniki pośrednie przechowywane w tabelach tymczasowych. Tymczasowość tych obiektów sprawia, że po zakończeniu procesu przetwarzania ślad po wykonanych transformacjach zanika, a zatem niemożliwe staje się ustalenie, skąd pochodzi dana wartość w tabeli wynikowej ani jakie operacje ją wytworzyły. Śledzenie takich zależności, czyli ustalanie, skąd pochodzi dana wartość i jakie operacje ją wytworzyły, jest przedmiotem dziedziny określanej w literaturze jako *data lineage* lub *data provenance* [7, 9]. Brak pełnej informacji o przepływie danych utrudnia audyt i kontrolę jakości danych.

Formalnie problem ten można wyrazić w kategoriach teorii grafów: graf zależności (ang. *lineage graph*) reprezentuje przepływ danych między obiektami bazy takimi jak tabele, widoki, zapytania oraz ich atrybutami. Gdy pewne węzły lub krawędzie zostaną utracone (np. z powodu usunięcia tabeli tymczasowej lub zastosowania funkcji definiowanej przez użytkownika), mówi się o zjawisku *broken lineage* (Boinski et al. 2025)[2]. Celem niniejszej pracy jest zastosowanie i ewaluacja skuteczności algorytmów wykrywania brakujących krawędzi w celu odtworzenia przerwanych połączeń w grafie lineage.

W sektorze finansowym banki są zobowiązane do wykazania, które transakcje źródłowe złożyły się na poszczególne pozycje raportów kwartalnych, przerwanie grafu zależności w warstwie ETL(*Extract, Transform, Load*) uniemożliwia przeprowadzenie takiego audytu [2]. Podobne konsekwencje niesie Rozporządzenie o Ochronie Danych Osobowych (RODO): brak pełnego grafu lineage sprawia, że dotarcie do wszystkich kopii danych konkretnej osoby jest czasochłonne lub niemożliwe, co może skutkować nałożeniem kar finansowych [2].

1.1 Cel i zakres pracy

Celem pracy jest zbadanie skuteczności istniejących algorytmów wykrywania brakujących krawędzi w grafach lineage. Zakres obejmuje trzy klasy metod:

- heurystyki grafowe (ang. *graph heuristics*),
- klasyczne metody uczenia maszynowego (ang. *Machine Learning, ML*),
- zanurzenia węzłów grafu (ang. *node embeddings*).

Eksperymenty przeprowadzono na zbiorze danych DLG-DG-23, obejmującym 18 zanonimizowanych grafów dwudzielnych ze środowiska Huawei Cloud (Chen et al. 2024) [3]. Przyjęto trzy scenariusze symulacji brakujących krawędzi, odzwierciedlające typowe przypadki broken lineage:

- **Scenariusz A** — symulacja ukrycia zależności przez tabelę pośrednią (staging): usunięcie krawędzi wychodzących z węzłów o roli pośredniej ($\text{in_df} > 0$ i $\text{out_df} > 0$),
- **Scenariusz B** — symulacja ukrycia zależności przez funkcję UDF: losowe usunięcie 20% krawędzi wychodzących z węzłów źródłowych ($\text{in_df} = 0$),
- **Scenariusz C** — symulacja ukrycia krawędzi do tabeli docelowej (data mart): usunięcie krawędzi wchodzących do węzłów o roli docelowej ($\text{out_df} = 0$).

Zakres pracy nie obejmuje odtworzenia rzeczywistego schematu bazy danych ani analizy treści zanonimizowanych danych.

1.2 Struktura pracy

Praca składa się z pięciu rozdziałów. Rozdział 2 zawiera przegląd pojęć podstawowych (data lineage, data provenance, broken lineage oraz wykrywanie krawędzi) a także omówienie wybranych prac z literatury i stosowanych metryk ewaluacji. Rozdział 3 opisuje problem badawczy, architekturę zaproponowanego rozwiązania, metodykę symulacji brakujących krawędzi oraz zaimplementowane algorytmy. Rozdział 4 prezentuje wyniki eksperymentów przeprowadzonych na zbiorze DLG-DG-23 wraz z ich analizą. Rozdział 5 zawiera wnioski końcowe, podsumowanie uzyskanych wyników oraz propozycje dalszych kierunków badań.

Rozdział 2

Podstawy teoretyczne i przegląd literatury

Niniejszy rozdział wprowadza pojęcia podstawowe niezbędne do zrozumienia dalszych części pracy. W sekcji 2.1 zdefiniowano pojęcia *data lineage* i *data provenance*. Sekcja 2.2 opisuje strukturę grafów lineage na przykładzie zbioru danych DLG-DG-23. Sekcja 2.3 omawia przyczyny niekompletności grafów, a sekcja 2.4 przedstawia problem wykrywania krawędzi. Sekcja 2.5 opisuje metryki ewaluacji, a sekcja 2.6 zawiera przegląd wybranej literatury.

2.1 Data lineage i data provenance

Pojęcia *data lineage* i *data provenance* są w literaturze stosowane zarówno zamiennie, jak i z różnicowaniem semantycznym [7]. W węższym ujęciu *data lineage* odnosi się do śledzenia przepływu informacji przez kolejne etapy przetwarzania, opisuje, przez jakie transformacje przeszły dane i jak zmieniała się ich postać. Z kolei *data provenance* kładzie nacisk na pochodzenie i autentyczność danych, rejestruje, skąd dane zostały pobrane i kto je wytworzył. W praktyce granica między tymi pojęciami jest płynna i większość opracowań traktuje je jako synonimy. Takie podejście przyjęto również w niniejszej pracy.

Znajomość historii danych ma istotne znaczenie praktyczne. W systemach raportowych możliwość odtworzenia ścieżki transformacji pozwala zlokalizować źródło błędnych wartości, bez tej informacji ręczna weryfikacja wielkich zbiorów danych jest nieefektywna. Innym przykładem jest bezpieczne usuwanie obiektów bazy: tabela pozornie niepowiązana z żadnym procesem może w rzeczywistości dostarczać danych do raportów końcowych, a jej usunięcie spowoduje utratę danych bez widocznego komunikatu błędu (Boinskiet al. 2025)[2].

2.2 Struktura grafu lineage w zbiorze DLG-DG-23

Zbiór danych DLG-DG-23, opracowany przez Chen et al. [3], zawiera 18 grafów lineage pobranych ze środowiska produkcyjnego Huawei Cloud. Dane zostały zanonimizowane — nie udostępniono schematów tabel. Grafy różnią się skalą: liczba węzłów waha się od 278 do 17 085.

Każdy graf zawiera trzy typy węzłów:

- **job** — zapytanie SQL lub zadanie ETL (ang. *Extract, Transform, Load*),
- **table** — tabela lub widok przechowujący dane,
- **field** — atrybut (kolumna) należący do tabeli.

Krawędzie grafu przyjmują dwa typy:

- **data_flow** — krawędź skierowana między tabelą a jobem lub w kierunku odwrotnym; reprezentuje przepływ danych,
- **parent_child** — krawędź między tabelą a jej atrybutem (polem); wyraża relację przynależności kolumny do tabeli.

Pełny graf DLG-DG-23 jest grafem heterogenicznym: zawiera węzły trzech typów połączone krawędziami dwóch typów. W niniejszej pracy predykcja krawędzi ogranicza się do podgrafu indukowanego przez węzły *job* i *table* powiązane krawędziami *data_flow*. Podgraf ten ma strukturę dwudzielną, każda krawędź łączy węzeł jednego typu z węzłem drugiego.

2.3 Niekompletność grafów lineage — broken lineage

Graf lineage bywa niekompletny z kilku powodów. Tabele tymczasowe, tworzone na potrzeby pojedynczej sesji lub procesu ETL, znikają po jego zakończeniu. Jeśli pośredniczyły w przepływie danych między dwoma trwałymi tabelami, znika wraz z nimi odpowiedni fragment grafu — krawędź reprezentująca tę zależność nigdy nie zostaje utrwalona w metadanych. Analogiczny skutek mają widoki zmaterializowane tworzone tymczasowo oraz funkcje definiowane przez użytkownika (ang. *User-Defined Function*, UDF), które opakowują logikę transformacji bez jawnego zapisu przepływu danych [9].

Zjawisko to, określane jako *broken lineage*, prowadzi do sytuacji, w której graf zawiera węzły bez połączeń lub z niepełnym sąsiedztwem, mimo że rzeczywista zależność między odpowiadającymi im obiektami istnieje. W niniejszej pracy problem broken lineage jest modelowany jako zadanie wykrywania brakujących krawędzi — każdej usuniętej krawędzi odpowiada para węzłów, dla której algorytm ma ocenić prawdopodobieństwo istnienia połączenia (Boinski et al. 2025)[2].

2.4 Wykrywanie brakujących krawędzi

Wykrywanie brakujących krawędzi (ang. *link prediction*) to zadanie uczenia maszynowego polegające na ocenie prawdopodobieństwa istnienia krawędzi między parą węzłów w grafie [5]. W klasycznym ujęciu problem ten pojawia się w grafach, w których część krawędzi nigdy nie została zaobserwowana. W niniejszej pracy przyjęto odmienne podejście: dysponując pełnymi grafami lineage, krawędzie są usuwane w sposób kontrolowany zgodnie z przyjętymi scenariuszami symulacji (rozdział 3), a zadaniem algorytmów jest ich odtworzenie.

W pracy zastosowano trzy klasy algorytmów:

1. **Heurystyki grafowe** — miary podobieństwa sąsiedztwa węzłów, takie jak Common Neighbors, indeks Jaccarda i miara Adamic-Adar [6]; nie wymagają uczenia, opierają się wyłącznie na strukturze grafu. Miary oparte na wspólnych sąsiadach wymagają adaptacji dla grafów dwudzielnych, w których węzły tego samego typu nie posiadają wspólnych sąsiadów bezpośrednich (omówiono w sekcji 3.3).
2. **Klasyczne metody ML** — Random Forest, RUSBoost i LightGBM; wykorzystują wektory cech opisujących pary węzłów, w tym cechy strukturalne grafu natomiast metryki podobieństwa nazw (np. Jaro-Winkler) nie mogą być zastosowane, ponieważ zbiór DLG-DG-23 używa zanonimizowanych identyfikatorów UUID.
3. **Zanurzenia sieciowe** (ang. *network embeddings*) — metoda Node2Vec, która odwzorowuje węzły grafu w przestrzeni wektorowej tak, by węzły o podobnym kontekście strukturalnym

były reprezentowane przez zbliżone wektory; wykrywanie krawędzi odbywa się przez klasyfikator (MLP) operujący na iloczynie Hadamarda embeddingów pary węzłów. Należy podkreślić, że Node2Vec nie jest grafową siecią neuronową (GNN) — uczy reprezentacji wyłącznie poprzez biaśed random walks, bez mechanizmu propagacji wiadomości (*message passing*). Rozszerzenie pracy o właściwą rodzinę GNN (np. R-GCN, HGT) wskazano w rozdziale podsumowującym jako kierunek dalszych badań.

2.5 Metryki ewaluacji

Do oceny jakości wykrywania krawędzi stosuje się standardowe metryki klasyfikacji binarnej. W niniejszej pracy przyjęto następujące metryki:

- **Precision** (precyzja) — frakcja poprawnie przewidzianych krawędzi spośród wszystkich krawędzi wskazanych przez model,
- **Recall** (czułość) — frakcja poprawnie wykrytych krawędzi spośród wszystkich rzeczywiście brakujących krawędzi,
- **F1** — średnia harmoniczna precyzji i czułości; główna miara porównawcza stosowana w pracy,
- **AUC-ROC** — pole pod krzywą ROC; mierzy zdolność modelu do rozróżnienia par połączonych od niepołączonych,
- **AUC-PR** — pole pod krzywą precyzja-czułość; szczególnie istotne przy nie zrównoważonych klasach, gdy liczba par niepołączonych wielokrotnie przewyższa liczbę rzeczywistych krawędzi.

Metryka F1 jest szczególnie istotna w warunkach nie zrównoważonych klas, charakterystycznych dla grafów rzadkich, gdzie liczba potencjalnych brakujących krawędzi wielokrotnie przewyższa liczbę krawędzi rzeczywistych [2, 5].

2.6 Wyniki systematycznego przeglądu literatury

Przeprowadzony systematyczny przegląd literatury (ang. *Systematic Literature Review*, SLR) pozwolił zidentyfikować siedem prac bezpośrednio odnoszących się do postawionych pytań badawczych. Spośród nich dwie mają bezpośrednie zastosowanie w niniejszej pracy: pierwsza stanowi jej główną inspirację i punkt odniesienia metodologicznego, druga dostarcza danych empirycznych wykorzystywanych w eksperymentach. Pozostałe prace pełnią rolę tła literaturowego i potwierdzają aktualność oraz zasadność podjętego problemu badawczego.

2.6.1 Leveraging machine learning techniques for discovering broken lineage links

Autorzy: Problem badawczy

Ciągłość grafów *data lineage* ulega przerwaniu, gdy w potokach ETL (ang. *Extract, Transform, Load*) wykorzystywane są obiekty pośrednie — tabele tymczasowe lub funkcje definiowane przez użytkownika (ang. *User-Defined Function*, UDF). Elementy te mają charakter tymczasowy i są usuwane po zakończeniu procesu przetwarzania. W rezultacie istniejące narzędzia klasy *data governance* tracą ślad przepływu danych i błędnie interpretują trwałą tabelę źródłową oraz tabelę docelową jako byty niezależne.

Zaproponowane rozwiązanie

Autorzy formułują problem rekonstrukcji przerwanych zależności jako binarną klasyfikację par obiektów bazodanowych. Podejście obejmuje następujące elementy:

- inżynierię cech opartą na ekstrakcji metadanych i analizie podobieństwa semantycznego nazw tabel oraz atrybutów z wykorzystaniem miary odległości Jaro–Winkler,
- klasyfikację z użyciem algorytmów Random Forest, RUSBoost, BalancedBagging oraz LightGBM,
- generowanie etykiet referencyjnych (*ground truth*) metodą rekurencyjnego zastępowania tabel tymczasowych ich trwałymi źródłami.

Ze względu na brak ogólnodostępnych zbiorów danych autorzy wygenerowali syntetyczne schematy baz danych z wykorzystaniem dużych modeli językowych (ang. *Large Language Model*, LLM): schemat systemu zarządzania ochroną zdrowia (model Grok) jako zbiór treningowy oraz schemat systemu finansowego (model Llama 3) jako zbiór testowy.

Analiza krytyczna

Najwyższą skuteczność spośród testowanych klasyfikatorów osiągnął algorytm RUSBoost — miara F1 wyniosła 0,79. Metoda wykazuje odporność na silne niezbalansowanie klas, typowe dla tego rodzaju problemu, gdzie liczba par bez relacji znacznie przewyższa liczbę par powiązanych. Istotnym ograniczeniem jest zależność jakości wykrywania od spójności konwencji nazewniczych stosowanych w bazie danych oraz konieczność ręcznej weryfikacji danych syntetycznych generowanych przez modele LLM.

Praca ta stanowi bezpośrednią motywację dla niniejszej pracy magisterskiej. Zastosowane przez autorów cechy topologiczne i modele klasyfikacji zostały zaadaptowane i rozszerzone, natomiast problem *broken lineage* — zdefiniowany w tej publikacji — przyjęto jako centralny problem badawczy.

2.6.2 An open dataset of data lineage graphs

Autorzy: Problem badawczy

Rozwój metod badawczych w obszarze *data governance* jest ograniczany przez brak ogólnodostępnych, rzeczywistych zbiorów danych. Przedsiębiorstwa nie udostępniają grafów *lineage* ze względu na poufność procesów biznesowych i wymogi bezpieczeństwa.

Zaproponowane rozwiązanie

Autorzy opublikowali zbiór DLG-DG-23 — pierwszy otwarty zestaw rzeczywistych grafów pochodzenia danych, charakteryzujący się następującymi właściwościami:

- źródło: środowisko produkcyjne Huawei Cloud (dane zanonimizowane),
- liczba grafów: 18, o zróżnicowanej wielkości w zakresie 278–17 085 węzłów,
- scenariusze zastosowań: infrastruktura chmurowa, obsługa klienta, analiza operacyjna,
- model grafu: węzły trzech typów — tabela, zadanie i pole — oraz relacje przepływu danych i hierarchii.

Analiza krytyczna

Unikalną wartość zbioru stanowi jego rzeczywiste pochodzenie przemysłowe, odróżniające go od danych generowanych syntetycznie. Zbiór umożliwia zastosowanie do różnorodnych zadań analitycznych: wykrywania anomalii, wizualizacji zależności oraz rekonstrukcji grafów. Ograniczeniem jest stosunkowo niewielka liczba grafów, utrudniająca trenowanie dużych modeli głębokich, a anonimizacja danych ogranicza dostępność atrybutów węzłów.

W niniejszej pracy zbiór DLG-DG-23 pełni rolę referencyjnego zbioru danych (*ground truth*) we wszystkich przeprowadzonych eksperymentach dotyczących rekonstrukcji grafów *lineage*.

2.6.3 Prace kontekstowe

Spośród pozostałych prac zidentyfikowanych w SLR na szczególną uwagę zasługuje metoda *Provenance Graph Kernel* (PGK) [5], która definiuje miary podobieństwa między grafami proveniencji i może stanowić punkt odniesienia przy ocenie jakości zrekonstruowanych grafów. Pozostałe prace — dotyczące proveniencji rekurencyjnych zapytań SQL [4], analizy logów systemowych [7], heterogenicznych grafowych sieci neuronowych (ang. *Graph Neural Network*, GNN) [8] oraz rozszerzonego *lineage* dla funkcji UDF [9] — tworzą szerszy kontekst literaturowy i potwierdzają aktualność problematyki odkrywania zależności w danych. Metody w nich opisane nie są bezpośrednio stosowane w niniejszej pracy, jednak wskazują kierunki możliwych rozszerzeń omówionych w rozdziale 5.

Rozdział 3

Problem i sposób jego rozwiązania

Niniejszy rozdział opisuje architekturę zaproponowanego rozwiązania, pochodzenie i charakterystykę danych, metodykę symulacji zjawiska broken lineage oraz szczegóły implementacji zrealizowanych algorytmów.

3.1 Charakterystyka danych

TABELA 3.1: Charakterystyka grafów zbioru DLG-DG-23.

ID	Węzły	Krawędzie	DATA_FLOW	Skala
DLG1	298		24	mały
DLG2	464		37	mały
DLG3	603		57	mały
DLG4	415		32	mały
DLG5	1 299		175	średni
DLG6	13 840		1 443	duży
DLG7	574		43	mały
DLG8	546		34	mały
DLG9	756		62	mały
DLG10	5 378		497	duży
DLG11	2 024		121	średni
DLG12	5 645		469	duży
DLG13	2 157		229	średni
DLG14	453		24	mały
DLG15	558		41	mały
DLG16	5 574		745	duży
DLG17	651		38	mały
DLG18	17 085		1 694	duży

Skala: mały < 1 000 węzłów; średni 1 000–5 000; duży > 5 000.
Liczba krawędzi DATA_FLOW to suma zbiorów train + val + test (podział 60/20/20).

3.2 Scenariusze symulacji broken lineage

3.3 Zaimplementowane algorytmy

3.3.1 Heurystyki grafowe

Heurystyki grafowe stanowią najprostszą klasę metod wykrywania krawędzi (*link prediction*). Wynik dla pary węzłów (u, v) wyznaczany jest wyłącznie na podstawie lokalnej lub globalnej struktury grafu bez etapu trenowania modelu. Metody te charakteryzują się niską złożonością obliczeniową i pełną interpretowalnością, co czyni je naturalnym punktem odniesienia (ang. *baseline*) przy porównywaniu bardziej zaawansowanych podejść [6].

Preferential Attachment

Miara *Preferential Attachment* (PA) opiera się na obserwacji, że w sieciach rzeczywistych nowe połączenia powstają z prawdopodobieństwem proporcjonalnym do stopnia istniejących węzłów. Oznacza to, że węzły o wielu sąsiadach z większym prawdopodobieństwem pozyskują kolejne połączenia [1]. Jako heurystyka wykrywania krawędzi PA została zastosowana przez Liben-Nowella i Kleinberga [6].

Dla pary węzłów (u, v) wynik PA definiuje się jako:

$$PA(u, v) = |N(u)| \cdot |N(v)|,$$

gdzie $N(u)$ oraz $N(v)$ oznaczają zbiory sąsiadów odpowiednio węzła u i węzła v , a $|\cdot|$ oznacza rozmiar zbioru, tj. stopień węzła w grafie.

W kontekście grafów *lineage* węzły grafu dwudzielnego DLG-DG-23 dzielą się na dwa typy: zadania przetwarzania danych (*job*) oraz tabele (*table*). Wysoki stopień węzła *job* oznacza, że dane zadanie czyta z wielu tabel źródłowych lub zapisuje do wielu tabel docelowych; wysoki stopień węzła *table* wskazuje na tabelę centralną. Para (u, v) , w której oba węzły mają duże stopnie, otrzymuje wysoki wynik PA, co interpretuje się jako sygnał potencjalnej zależności *data_flow*.

Zasadnicze ograniczenie PA polega na tym, że miara ignoruje pokrycie sąsiedztw: dwa węzły mogą mieć duże stopnie, lecz nie posiadać żadnych wspólnych sąsiadów, a mimo to otrzymać wysoki wynik. W grafach dwudzielnych takich jak DLG-DG-23 węzły tego samego typu (*job-job* lub *table-table*) nigdy nie mają wspólnych sąsiadów bezpośrednich, dlatego miary oparte na wspólnych sąsiadach (Common Neighbors, Jaccard, Adamic-Adar) wymagają adaptacji do struktury dwudzielnej. PA nie wymaga takiej adaptacji, obliczany jest bezpośrednio z stopni węzłów co stanowi jego przewagę praktyczną w omawianym scenariuszu, kosztem mniejszej precyzji wykrywania w porównaniu z miarami uwzględniającymi topologię sąsiedztwa.

3.3.2 Klasyczne metody uczenia maszynowego

3.3.3 Zanurzenia sieciowe — Node2Vec

3.4 Architektura rozwiązania

Rozdział 4

Ocena rozwiązania

Niniejszy rozdział przedstawia wyniki eksperymentów przeprowadzonych zgodnie z metodologią opisaną w rozdziale 3. Dla każdego z trzech scenariuszy symulacji oraz każdego z trzech algorytmów podano wartości metryk F1, Precision, Recall i AUC-ROC uśrednione po wszystkich 18 grafach zbioru DLG-DG-23.

4.1 Wyniki eksperymentów

4.2 Analiza wyników

Rozdział 5

Uwagi końcowe

Niniejszy rozdział zawiera podsumowanie uzyskanych wyników, wnioski dotyczące skuteczności zastosowanych algorytmów oraz propozycje dalszych kierunków badań.

5.1 Podsumowanie

5.2 Kierunki dalszych badań

Literatura

- [1] Albert-László Barabási and Réka Albert. Emergence of Scaling in Random Networks. *Science*, 286(5439):509–512, 1999.
- [2] Paweł Boiński, Witold Andrzejewski, Miłosz Grocholewski, Tobiasz Gruszczyński, and Robert Wrembel. Leveraging machine learning techniques for discovering broken lineage links between database objects. In *Proceedings of the 33rd International Conference on Information Systems Development (ISD2025)*, Belgrade, Serbia, 2025.
- [3] Yunpeng Chen, Ying Zhao, Xuanjing Li, Jiang Zhang, Jiang Long, and Fangfang Zhou. An open dataset of data lineage graphs for data governance research. *Visual Informatics*, 8:1–5, 2024.
- [4] Benjamin Dietrich, Tobias Müller, and Torsten Grust. Data Provenance for Recursive SQL Queries. In *Proceedings of the 14th Workshop on the Theory and Practice of Provenance (TaPP '22)*, Philadelphia, PA, USA, 2022. ACM.
- [5] David Kohan Marzagão, Trung Dong Huynh, Ayah Helal, Sean Baccas, and Luc Moreau. Provenance Graph Kernel. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 2023.
- [6] David Liben-Nowell and Jon Kleinberg. The Link-Prediction Problem for Social Networks. *Journal of the American Society for Information Science and Technology*, 58(7):1019–1031, 2007.
- [7] Fotis Psallidas, Ashvin Agrawal, Chandru Sugunan, Khaled Ibrahim, Konstantinos Karanasos, Jesús Camacho-Rodríguez, Avriella Floratou, Carlo Curino, and Raghu Ramakrishnan. ONEPROVENANCE: Efficient Extraction of Dynamic Coarse-Grained Provenance From Database Query Event Logs. *CoRR*, abs/2210.14047, 2022. Technical Report; deployed in Microsoft Purview.
- [8] Dibin Shan, Xuehui Du, Wenjuan Wang, Na Wang, and Aodi Liu. KPI-HGNN: Key Provenance Identification Based on a Heterogeneous Graph Neural Network for Big Data Access Control. *Information Sciences*, 2023.
- [9] Masaya Yamada, Hiroyuki Kitagawa, Toshiyuki Amagasa, and Akiyoshi Matono. Augmented lineage: traceability of data analysis including complex UDF processing. *The VLDB Journal*, 32:963–983, 2023.



© 2026 Maria Nowicka

Instytut Informatyki, Wydział Informatyki i Telekomunikacji
Politechnika Poznańska

Skład przy użyciu systemu $\text{\LaTeX}$ na platformie Overleaf.